

تمرین عملی و سناریومحور شبکه‌های عصبی کانولوشنی

تشخیص ارقام با دیتاست SVHN

زمان پیشنهادی: ۳ ساعت | PyTorch | اجرا روی CPU

سناریوی تمرین

در این تمرین قرار است یک شبکه عصبی کانولوشنی ساده بسازید که رقم موجود در یک تصویر را تشخیص دهد. به‌جای اینکه از ابتدا کل شبکه را بسازیم، مسیر داده را مرحله‌به‌مرحله دنبال می‌کنیم. در هر گام ابتدا خروجی واقعی را مشاهده می‌کنید و سپس فقط درباره همان اتفاق پاسخ می‌دهید.

مسیر کلی تمرین: آشنایی با SVHN → بررسی یک ReLU → Conv1 → Batch و Pooling → Conv2 → Forward کامل → Padding و Feature Maps → Test → Training → Stride

SVHN مخفف Street View House Numbers است. این دیتاست شامل تصاویر رنگی ارقام واقعی است که از تصاویر شماره‌های موجود در محیط شهری استخراج شده‌اند. در این تمرین با تصاویر 32×32 و سه کاناله RGB کار می‌کنیم و مسئله ما یک طبقه‌بندی ۱۰ کلاسه است؛ یعنی مدل باید برای هر تصویر یکی از ارقام ۰ تا ۹ را پیش‌بینی کند.

نسخه torchvision این دیتاست دارای مجموعه‌های Train و Test آماده است. بنابراین در این تمرین نیازی به پاک‌سازی داده، Resize، Crop یا Data Augmentation ندارید. تنها تبدیل موردنیاز ToTensor() است تا تصویر به Tensor قابل استفاده در PyTorch تبدیل شود.

هدف از انتخاب SVHN: تصاویر کوچک و رنگی هستند، با CNN ساده قابل آموزش‌اند و بدون ورود به مباحث پیش‌پردازش، می‌توان همان مفاهیم تدریس‌شده را روی یک دیتاست جدید تمرین کرد.

گام ۱ — داده را بارگذاری و بررسی کنید

ابتدا Dataset و DataLoader را بسازید:

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

from torchvision import datasets, transforms
from torch.utils.data import DataLoader

transform = transforms.ToTensor()

train_dataset = datasets.SVHN(
    root="./data",
    split="train",
    download=True,
    transform=transform
)

test_dataset = datasets.SVHN(
    root="./data",
```

```
split="test",
download=True,
transform=transform
)

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False
)
```

یک Batch از داده‌های Train بردارید و Shape آن را بررسی کنید:

```
images, labels = next(iter(train_loader))

print("images:", images.shape)
print("labels:", labels.shape)
```

چند تصویر را هم نمایش دهید:

```
fig, axes = plt.subplots(1, 5, figsize=(10, 2))

for i in range(5):
    img = images[i].permute(1, 2, 0)
    axes[i].imshow(img)
    axes[i].set_title(f"Label: {labels[i].item()}")
    axes[i].axis("off")

plt.tight_layout()
plt.show()
```

الف) Shape مربوط به images و labels را ثبت کنید.

ب) در images.shape، عدد ۶۴ چه چیزی را نشان می‌دهد؟

ج) کدام عدد نشان می‌دهد تصاویر رنگی و سه‌کاناله هستند؟

د) اندازه فضایی هر تصویر چند در چند است؟

ه) Shape یک تصویر بدون Batch چیست؟

نتیجه این گام: اکنون می‌دانید هر نمونه ورودی CNN یک تصویر RGB با Shape برابر [۳۲, ۳۲, ۳] است.

گام ۲ — مدل پایه را تعریف کنید و فقط Conv۱ را بررسی کنید

مدل را از ابتدا به‌صورت کامل تعریف می‌کنیم تا در مراحل بعد دوباره مجبور به تغییر ساختار کلاس نباشید. فعلاً مدل را Train نکنید؛ فقط خروجی لایه‌ها را بررسی می‌کنیم.

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Conv2d(
            in_channels=3,
            out_channels=16,
            kernel_size=3,
            padding=1
        )

        self.pool = nn.MaxPool2d(
            kernel_size=2,
            stride=2
        )

        self.conv2 = nn.Conv2d(
            in_channels=16,
            out_channels=32,
            kernel_size=3,
```

```

        padding=1
    )

    self.fc = nn.Linear(32 * 8 * 8, 10)

def forward(self, x):
    x = self.conv1(x)
    x = torch.relu(x)
    x = self.pool(x)

    x = self.conv2(x)
    x = torch.relu(x)
    x = self.pool(x)

    x = torch.flatten(x, 1)
    x = self.fc(x)

    return x

model = CNN()

```

حالا فقط Conv۱ را روی همان Batch اجرا کنید:

```

x = model.conv1(images)

print("Before Conv1:", images.shape)
print("After Conv1 :", x.shape)

```

الف) تعداد Channelها قبل از Conv۱ و بعد از Conv۱ را ثبت کنید.

ب) در تعریف Conv۲d(۳, ۱۶, ...)، عدد ۳ با کدام بخش Shape ورودی ارتباط دارد؟

ج) عدد ۱۶ با کدام بخش Shape خروجی ارتباط دارد؟

د) Conv۱ برای هر تصویر چند Feature Map تولید کرده است؟

هـ) ارتباط بین تعداد Filterهای Conv۱، تعداد Feature Mapها و Output Channelها را در یک جمله توضیح دهید.

گام ۳ — خروجی Conv۱ را از ReLU و MaxPooling عبور دهید

از همان Batch استفاده کنید و مسیر را تا Pool۱ ادامه دهید:

```
x = model.conv1(images)
print("After Conv1:", x.shape)

before_relu = x
x = torch.relu(x)
print("After ReLU :", x.shape)

x = model.pool(x)
print("After Pool :", x.shape)
```

الف) Shape قبل و بعد از ReLU را مقایسه کنید. آیا ReLU Shape را تغییر داده است؟

ب) چند مقدار منفی را در before_relu پیدا کنید و مقدار همان عناصر را بعد از ReLU مشاهده کنید. چه اتفاقی افتاده است؟

ج) بعد از MaxPooling، Height و Width چه تغییری کرده‌اند؟

د) آیا تعداد Channel‌ها بعد از Pooling تغییر کرده است؟

ه) با توجه به خروجی خودتان، تفاوت نقش ReLU و MaxPooling را کوتاه توضیح دهید.

گام ۴ — Conv۲ را به مسیر اضافه کنید

خروجی Pool۱ اکنون ۱۶ Channel دارد. همان خروجی را وارد Conv۲ کنید:

```
x = model.conv1(images)
x = torch.relu(x)
x = model.pool(x)

print("Input to Conv2 :", x.shape)

x = model.conv2(x)

print("Output of Conv2:", x.shape)
```

الف) ورودی Conv۲ چند Channel دارد؟

ب) چرا in_channels در Conv۲ برابر ۱۶ تعریف شده است؟

ج) چرا خروجی Conv۲ دارای ۳۲ Channel است؟

د) این ۳۲ Channel کانال‌های رنگی هستند یا Feature Map های ساخته شده توسط Conv۲؟

نکته: سه Channel ورودی اولیه مربوط به RGB هستند؛ اما Channel های لایه های میانی شبکه Feature Map هستند.

گام ۵ — Forward کامل مدل را مشاهده کنید

اکنون مسیر کامل مدل تعریف شده است. برای اینکه ببینید داده در طول شبکه چه شکلی پیدا می کند، یک بار همان مراحل forward را خارج از مدل اجرا کرده و Shape هر مرحله را چاپ کنید.

```
x = images
print("Input :", x.shape)

x = model.conv1(x)
print("Conv1 :", x.shape)

x = torch.relu(x)
x = model.pool(x)
print("Pool1 :", x.shape)

x = model.conv2(x)
print("Conv2 :", x.shape)

x = torch.relu(x)
x = model.pool(x)
print("Pool2 :", x.shape)

x = torch.flatten(x, 1)
print("Flatten :", x.shape)

x = model.fc(x)
```

```
print("Linear :", x.shape)
```

مرحله	Shape مشاهده شده
Input	
Conv۱	
Pool۱	
Conv۲	
Pool۲	
Flatten	
Linear	

الف) در کدام قسمت‌های شبکه تعداد Channel‌ها تغییر کرده است؟

ب) در کدام قسمت‌ها Height و Width کاهش پیدا کرده‌اند؟

ج) Flatten چه تغییری در شکل داده ایجاد کرده است؟

د) چرا خروجی نهایی Linear برای هر تصویر شامل ۱۰ مقدار است؟

گام ۶ — اثر Padding و Stride را روی Conv۱ بررسی کنید

اکنون که رفتار مدل پایه را دیده‌اید، فقط برای یک آزمایش کوتاه Conv۱ را در سه حالت مختلف اجرا کنید. نیازی به تغییر مدل اصلی نیست.

```
conv_a = nn.Conv2d(3, 16, 3, padding=1, stride=1)
conv_b = nn.Conv2d(3, 16, 3, padding=0, stride=1)
conv_c = nn.Conv2d(3, 16, 3, padding=1, stride=2)
```

```
print("A:", conv_a(images).shape)
print("B:", conv_b(images).shape)
print("C:", conv_c(images).shape)
```


Output Shape	تنظیم	حالت
	padding=۱ , stride=۱	A
	padding=۰ , stride=۱	B
	padding=۱ , stride=۲	C

الف) وقتی Padding از ۱ به ۰ تغییر کرد، Width و Height چه تغییری کردند؟

ب) وقتی Stride از ۱ به ۲ تغییر کرد، Width و Height چه تغییری کردند؟

ج) با توجه به همین سه خروجی، نقش Padding و Stride را با زبان خودتان توضیح دهید.

گام ۷ — مدل را آموزش دهید

اکنون که مسیر Forward را می‌شناسید، مدل را برای چند Epoch کوتاه آموزش دهید. برای مناسب ماندن زمان اجرا روی CPU می‌توانید از بخشی ثابت از داده‌های Training استفاده کنید.

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

```
weights_before = model.conv1.weight.detach().clone()
```

Training Loop را مطابق الگوی کلاس اجرا کنید و Loss هر Epoch را ثبت کنید.

Epoch	Training Loss
۱	
۲	
۳	

الف) Loss از Epoch اول تا آخر چه تغییری کرده است؟

ب) `loss.backward()` در فرایند Training چه کاری انجام می‌دهد؟

ج) `optimizer.step()` چه کاری انجام می‌دهد؟

بعد از پایان Training، وزن‌های Conv۱ را دوباره ذخیره کنید:

```
weights_after = model.conv1.weight.detach().clone()

print(
    torch.allclose(
        weights_before,
        weights_after
    )
)
```

د) آیا Weight‌های Conv۱ بعد از Training تغییر کرده‌اند؟

ه) آیا `kernel_size` یا تعداد Filterهای Conv۱ در طول Training تغییر کرده‌اند؟

و) با توجه به مشاهده خودتان، وقتی می‌گوییم Filterهای CNN یاد گرفته می‌شوند، دقیقاً چه چیزی در آن‌ها به‌روزرسانی می‌شود؟

گام ۸ — مدل را روی Test Set ارزیابی کنید

مدل را روی `test_loader` ارزیابی و Test Accuracy را محاسبه کنید.

Test Accuracy:

حالا واقعاً یک Batch از Test Set بردارید:

```
test_images, test_labels = next(iter(test_loader))

model.eval()
with torch.no_grad():
    logits = model(test_images)
    predictions = logits.argmax(dim=1)

print("True labels :", test_labels[:10])
```

```
print("Predictions :", predictions[:10])
```

چند نمونه را همراه با برچسب واقعی و پیش‌بینی مدل نمایش دهید:

```
fig, axes = plt.subplots(1, 5, figsize=(10, 2))

for i in range(5):
    img = test_images[i].permute(1, 2, 0)
    axes[i].imshow(img)
    axes[i].set_title(
        f"T:{test_labels[i].item()} / P:{predictions[i].item()}"
    )
    axes[i].axis("off")

plt.tight_layout()
plt.show()
```

الف) چند نمونه را پیدا کنید که مدل درست تشخیص داده است.

ب) آیا نمونه‌ای وجود دارد که مدل اشتباه تشخیص داده باشد؟

ج) Test Accuracy چه اطلاعاتی درباره عملکرد مدل روی داده‌های Test به شما می‌دهد؟

گام ۹ — Feature Map های Conv۱ را مشاهده کنید

برای آخرین مرحله، یکی از تصاویر Test را انتخاب کنید و خروجی Conv۱ را مشاهده کنید. این بار از test_images استفاده می‌کنیم تا متن تمرین و کد دقیقاً با هم منطبق باشند.

```
image = test_images[0].unsqueeze(0)

with torch.no_grad():
    feature_maps = model.conv1(image)

print(feature_maps.shape)
```

تعدادی از Feature Map ها را نمایش دهید:

```
fig, axes = plt.subplots(4, 4, figsize=(8, 8))

for i, ax in enumerate(axes.flat):
    ax.imshow(
        feature_maps[0, i].cpu(),
        cmap="gray"
    )
    ax.set_title(f"Map {i+1}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```

الف) برای این یک تصویر چند Feature Map تولید شده است؟

ب) چرا این تعداد با تعداد Filterهای Conv₁ برابر است؟

ج) با وجود یکسان بودن تصویر ورودی، چرا Feature Mapهای مختلف ظاهر یکسانی ندارند؟

د) این تفاوت چه ارتباطی با Weightهای متفاوت Filterها دارد؟

جمع‌بندی — داستان یک تصویر SVHN

مرحله	آنچه مشاهده کردید
ورودی	تصویر RGB با اندازه ۳۲×۳۲
Conv ₁	تولید ۱۶ Feature Map
ReLU	تغییر مقادیر بدون تغییر Shape
Pool ₁	کاهش اندازه فضایی
Conv ₂	تولید ۳۲ Feature Map
Pool ₂	کاهش دوباره اندازه فضایی
Flatten	تبدیل Feature Mapها به بردار

تولید ۱۰ خروجی	Linear
به‌روزرسانی Weight های قابل یادگیری	Training

در چند جمله، مسیر یک تصویر SVHN را از ورود به CNN تا پیش‌بینی رقم توضیح دهید. در توضیح خود از واژه‌های Linear و Channel، Filter، Feature Map، ReLU، MaxPooling، Flatten استفاده کنید.